

Reviewer Pack — Galois Concept Lattice Height Lower-Bounds Monotone KW Depth

Entry bba09a0a4ff8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 10:25:04 UTC

Galois Concept Lattice Height Lower-Bounds Monotone KW Depth

Entry ID: bba09a0a4ff8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 10:25:04 UTC

1. Conjecture

Field A (mathematical branch): Formal Concept Analysis (Wille 1982; Ganter-Wille 1999): Galois concept lattices $L(K)$ of binary contexts $K=(G,M,I)$, accessed via the NextClosure algorithm and the chain-height invariant $h(L(K)) := \text{length of the longest concept chain}$ — a poly-time order-theoretic feature of bipartite incidences essentially absent from circuit/proof complexity. **Field B** (complexity object): Karchmer-Wigderson monotone formula depth $D_+(f)$ of monotone Boolean $f: \{0,1\}^n \rightarrow \{0,1\}$, equivalently the optimal monochromatic-rectangle depth of the KW relation on $\text{Min}(f) \times \text{Max}(f)$ with answers in $[n]$ (Karchmer-Wigderson 1990; Raz-Wigderson; Tzameret).

Statement:

For every monotone Boolean $f: \{0,1\}^n \rightarrow \{0,1\}$ with $|\text{Min}(f)| \geq 2$ and $|\text{Max}(f)| \geq 2$, define the FCA context $K_f := (\text{Min}(f), \text{Max}(f) \times [n], I_f)$ with $I_f(x, (y,i)) = 1$ iff $x_i = 1$ and $y_i = 0$; let $L(K_f)$ be its Galois concept lattice and $h(K_f)$ the length of its longest concept chain. Then the monotone Karchmer-Wigderson formula depth obeys $D_+(f) \geq \lceil \log_2 h(K_f) \rceil$. A single monotone f exhibiting $h(K_f) > 2^{D_+(f)}$ falsifies the conjecture.

Rationale (proposer's reasoning):

Each formal concept is a maximal KW-consistent rectangle of (true-input, false-input $\times$ answer) pairs; a chain of concepts is a strict refinement sequence of such rectangles, so any KW protocol must spend at least $\lceil \log_2 h(K_f) \rceil$ rounds to resolve the chain. FCA chain-height is unaffected by lifting tricks that defeat rank/discrepancy bounds, so it bypasses algebrization while retaining a poly-time proxy. The bound is tight for AND_n and OR_n ($h=1$) and grows with implicant-clause incidence richness, where existing rank/log-rank certificates plateau.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ef7007ff988f48a7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 random monotone Boolean functions f on $n \in \{4, 5, 6\}$ with $|\text{Min}(f)| \geq 2$ and $|\text{Max}(f)| \geq 2$, compute exact monotone KW depth $D_+(f)$ and Galois concept lattice height $h(K_f)$. Conjecture is supported iff $D_+(f) \geq \lceil \log_2 h(K_f) \rceil$ holds on $\geq 29/30$ seeds; any single strict violation falsifies it.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.85	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Karchmer-Wigderson monotone formula depth concept lattice - formal concept analysis Boolean function complexity Galois lattice - monotone circuit depth lower bound rectangle minterm maxterm lattice height

Top relevant hits considered: - [<http://arxiv.org/abs/0710.4339v1>] Heavy-Quark Masses from the Fermilab Method in Three-Flavor Lattice QCD - [<http://arxiv.org/abs/1111.0981v1>] Form factors for B to $K\ell\ell$ semileptonic decay from three-flavor lattice QCD - [<http://arxiv.org/abs/2107.05128v2>] Karchmer-Wigderson Games for Hazard-free Computation - [<http://arxiv.org/abs/1210.2401v1>] Distributed Formal Concept Analysis Algorithms Based on an Iterative MapReduce Framework - [<http://arxiv.org/abs/2311.04204v3>] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def NextClosure(K_f, (Min_f, Max_f)):
    n = len(Max_f)
    closure = set()
    for x in Min_f:
        closure.add(x)
    while True:
        new_closure = set()
        for y in Max_f:
            if all(x[i] == (y[0][i], y[1][i]) for i in range(n)):
                new_closure.add(y)
        if new_closure.issubset(closure):
            break
        closure.update(new_closure)
```

```

    return closure

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([4, 5, 6])
    implicants = [frozenset(random.sample(range(n), k)) for _ in range(2 + random.randint(0, 4))]
    f = {tuple(sorted(x)): all(i in x for i in y) for x in implicants for y in implicants}

    Min_f = set()
    Max_f = set()
    for x in product([0, 1], repeat=n):
        if all(f[x] == (i in x for i in range(n)) for i in range(n)):
            Min_f.add(x)
        if all(not f[x] == (i in x for i in range(n)) for i in range(n)):
            Max_f.add(x)

    K_f = (Min_f, Max_f)
    H = NextClosure(K_f, K_f)
    h_K_f = max(len(path) for path in longest_path(H))

    D_plus_f = monotone_kw_depth(f)

    conjecture_holds = D_plus_f >= math.ceil(math.log2(h_K_f))
    counterexample = "" if conjecture_holds else f"Counterexample found with n={n}, Min(f)={Min_f}"

    return {
        "metric_name": "KW Depth",
        "metric_value": D_plus_f,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

def monotone_kw_depth(f):
    n = len(next(iter(f)))
    memo = {}

    def dp(x, y):
        if (x, y) in memo:
            return memo[(x, y)]
        if x == y:
            return 0
        if not any(f[x] for i in range(n)):
            return float('inf')
        if all(f[x] for i in range(n)):
            return 1
        min_depth = float('inf')
        for i in range(n):
            if f[x][i]:
                new_y = y[:i] + (0,) + y[i+1:]
                min_depth = min(min_depth, dp(x, new_y) + 1)
            else:
                new_y = y[:i] + (1,) + y[i+1:]
                min_depth = min(min_depth, dp(x, new_y) + 1)
        memo[(x, y)] = min_depth
        return min_depth

```

```

D_plus_f = float('inf')
for x in product([0, 1], repeat=n):
    for y in product([0, 1], repeat=n):
        if all(f[x] == (i in x for i in range(n)) for i in range(n)):
            D_plus_f = min(D_plus_f, dp(x, y))
return D_plus_f

def longest_path(graph):
    n = len(graph)
    dist = [[float('inf')] * n for _ in range(n)]
    for i in range(n):
        dist[i][i] = 0
    for u, v in graph:
        dist[u][v] = 1

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    paths = []
    for start in range(n):
        path = [start]
        current = start
        while len(path) < n and dist[current][path[-1]] != float('inf'):
            next_node = None
            min_dist = float('inf')
            for neighbor in range(n):
                if neighbor not in path and dist[current][neighbor] + dist[neighbor][path[-1]] ==
                    if dist[current][neighbor] < min_dist:
                        min_dist = dist[current][neighbor]
                        next_node = neighbor
            if next_node is None:
                break
            path.append(next_node)
            current = next_node
        paths.append(path)

    return paths

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 1 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):

```

```

    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample={r['counterexample']}\n first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47b39da1.py", line 147, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47b39da1.py", line 122, in run_trial
    new_closure = NextClosure(K_f, (Min_f, Max_f))
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47b39da1.py", line 98, in NextClosure
    if all(x[i] == (y[0][i], y[1][i]) for i in range(n)):
       ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47b39da1.py", line 98, in <genexpr>
    if all(x[i] == (y[0][i], y[1][i]) for i in range(n)):
       ~~~~~^
TypeError: 'set' object is not subscriptable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` ('set' object is not subscriptable) inside `NextClosure` before producing any `RESULT` line, so no seeds were evaluated against the pre-registered criterion. | next: Fix the `NextClosure` implementation to represent attribute sets as indexable/sorted structures (e.g., `frozenset` converted to sorted tuple, or bitmask) and re-run the 30-seed trial.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	316613
2	preregistration	claude_max	opus	0	0	5533

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	claude_max	opus	0	0	2995
4	novelty	claude_max	opus	0	0	10394
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15882
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15008
7	judge	claude_max	opus	0	0	4824

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 371249 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/bba09a0a4ff8.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/bba09a0a4ff8.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/bba09a0a4ff8.tar.gz (if generated)